

Project Collaboration Using Git

About Git

- Version control
- Tracks code change

Note

Git must be install in your systems

Git has 3 branches:

- test
 - dev
 - production
-

Github Setup

- Initialize a repo
- Commands

```
git init # initialize the repo
git add . # add all
git commit -m "mmm" # m is for message
git branch -M <branch_name> # M is for initialize a branch
git remote add origin <url> # input the url from github repo that has
already been initialize
git push origin -u <branch_name>
```

- Collaborator accept invite via email.
-

Cloning Repository

Branching

```
git branch # display branch
git branch <branchname> # create branch
git checkout <branchname> # change branch
git checkout -b <branchname> # checkout new branch
```

Saving Code Changes

- Every repository must have a `.gitignore` file.
- Uses of `.gitignore`
 - To exclude storage files we dont need
 - To exclude package installations (node_modules, vender, etc)
 - To exclude `.env`

```
# push procedure
git init # initialize the repo
git add . # add all
git commit -m "mmm" # m is for message
git push
# Rename lastes commit msg
git commit --amend -m "your correct msg"
# Rename any commit message at any timeline (5 here is last 5 commit)
git rebase -i HEAD-5
# Rename commit using the head hash
git rebase -i <hash>
# pull latest code of the branch
git pull
```

Merging

- Go to branch you want to merge with
- Merge with your development branch, ex: Merge a feature in a development branch to main branch
--> Current branch: main

Stashing

```
# Stash changes in files
git stash
# Stash include newly added file (untracked files)
git stash -u
# Show list of stashes
git stash list
# Applying stash and keep in list
git stash apply
# Applying latest stash and remove it in list (One time applying the stash)
git stash pop
```

```
# Apply stash from list
git stash apply stash@{number}
```

Merge Conflicts

| When does merge conflict happen?

- Two branches edited the same line differently
- Git does not know which change to keep Conflict markers in the file:
- <<<<<<<<< your current branch changes
- ===== divider between changes
- >>>>>>>> incoming branch changes How to resolve:
- Open the conflicted file
- Decide which change to keep
- Complete Merge

Pull Request & Review

Branch Protection Rules

Add class branch protection rule

WorkFlow

| Common workflow practices

- Single Branch
 - All changes committed directly to main
 - Good for solo projects or prototypes
- Feature Branch
 - Each feature has its own branch
 - Merged into main when complete
- Gitflow (Three branch)
 - Main: production
 - Testing: QA before release
 - Development: active development

- Pull request & Code Review
 - Applies to any workflow
 - Branch protection enforces the review process
 - Approved before any merge into main
- Versioning
 - Tag main branch at each release point
 - Ex: v1.0.0.0

```
# Go to main branch (Finalized)
git checkout main
git tag v1.0.0.0
git push origin v1.0.0
git checkout v1.0.0.0
```